FederHub Platform User Stories

Overview

This document presents the unified user stories for the FederHub project, focusing on the key personas: Platform Admin, ML Engineer (Federation Owner), and Client Operator. The platform supports privacy-preserving machine learning through decentralized orchestration, secure data handling, and metric monitoring.

Structured under 5 epics, each story adheres to the INVEST framework, MOSCOW prioritization, T-shirt sizing, and the 3 C's for agile development.

Epic 1: Platform Administration and Identity Management

Focus: Managing organizations, enforcing global security, and ensuring strict access control across the FederHub SaaS platform.

Story 1.1: As a Platform Admin, I want to create organizations and user accounts, so that I can securely onboard new hospitals or banks to the platform.

- **Acceptance Criteria:**
  - Given the admin is logged into the web interface, when they input new client details, then a secure user account is created.
  - Given an account is created, then the database generates valid credentials for the client.
- **Priority:** Must
- **Sizing:** S
- **3 C's:** Card: This story. Conversation: Discuss the database schema in PostgreSQL for user accounts. Confirmation: Verify successful account creation and credential assignment.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 1.2: As a Platform Admin, I want to deactivate organizations or users, so that I can immediately revoke access if a client withdraws from the study.

- **Acceptance Criteria:**
  - Given the admin views the organization list, when they select deactivate, then the client's access is immediately revoked.
  - Given a deactivated client attempts to log in, then the server rejects their credentials.
- **Priority:** Must
- **Sizing:** S

- **3 C's:** Card: This story. Conversation: Define how active JWT tokens are invalidated. Confirmation: Test login failure for deactivated accounts.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 2: Experiment Orchestration and Model Registry

Focus: Allowing ML Engineers to set up the mathematical foundation and orchestration parameters for the federated learning jobs.

Story 2.1: As an ML Engineer, I want to upload a blank, untrained model architecture to the Model Registry, so that there is a starting point for the training process.

- **Acceptance Criteria:**
  - Given the ML Engineer is on the dashboard, when they upload a model file (e.g., a CNN for images), then it is securely stored in the Model Registry.
  - Given the model is uploaded, then it can be tagged with tracking notes.
- **Priority:** Must
- **Sizing:** M
- **3 C's:** Card: This story. Conversation: Determine supported model types like PyTorch or TensorFlow. Confirmation: Verify the file is successfully saved in the registry.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 2.2: As an ML Engineer, I want to create a federated training job, so that I can define the number of rounds and local epochs for the experiment.

- **Acceptance Criteria:**
  - Given the model is selected, when the ML Engineer enters hyperparameters, then the system saves the job configuration.
  - Given the configuration is saved, then the Celery/Redis task orchestration queue prepares for round management.
- **Priority:** Must
- **Sizing:** M
- **3 C's:** Card: This story. Conversation: Map the exact configuration fields required for the Web UI. Confirmation: Check database to ensure job configurations are saved correctly.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 3: Client Onboarding and Strict Data Privacy

Focus: The client operator's local experience, ensuring sensitive data remains siloed behind firewalls.

Story 3.1: As a Client Operator, I want to log into the Desktop Application, so that I can receive a secure token to authenticate with the central server.

- **Acceptance Criteria:**
  - Given the user launches the Electron.js desktop wrapper, when they input credentials, then the FastAPI server verifies them.
  - Given verification passes, then the client app securely receives and stores a JWT (JSON Web Token).
- **Priority:** Must
- **Sizing:** S
- **3 C's:** Card: This story. Conversation: Define local token storage mechanisms. Confirmation: Validate successful login and token generation.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 3.2: As a Client Operator, I want to use a "Browse" button to select my local data folder, so that it is locked as a read-only Docker volume without uploading to the internet.

- **Acceptance Criteria:**
  - Given the user is logged in, when they click "Browse", then a local file explorer appears.
  - Given the folder is selected, then the app locks it as a read-only volume and no raw data leaves the local machine.
- **Priority:** Must
- **Sizing:** L
- **3 C's:** Card: This story. Conversation: Define Docker container sandbox security rules.Confirmation: Monitor network traffic to prove zero raw data is transmitted.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 4: Local Execution and Secure Weight Transfer

Focus: Activating the training process, mathematically erasing raw data visibility, and communicating with the central hub.

Story 4.1: As a Client Operator, I want to click a "Start Training" trigger, so that my local machine trains the model and flushes the raw data from memory.

- **Acceptance Criteria:**
  - Given the ML Engineer has started the global job, when the client clicks start, then the Python script converts local data into mathematical tensors.
  - Given local training epochs complete, then raw data is flushed, leaving only mathematical arrays of weights and gradients.
- **Priority:** Must

- **Sizing:** M
- **3 C's:** Card: This story. Conversation: Ensure memory wiping protocols are strictly enforced. Confirmation: Verify the memory state post-training.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 4.2: As a Client Operator, I want the system to stream only updated weights back to the server, so that I contribute to the global model securely.

- **Acceptance Criteria:**
    - Given the mathematical arrays are ready, when transmission begins, then they are serialized using Protocol Buffers (Protobuf).
    - Given the transfer occurs, then it streams over a high-speed, compressed gRPC channel.
- **Priority:** Must
- **Sizing:** L
- **3 C's:** Card: This story. Conversation: Configure Flower or FedML libraries for bi-directional gRPC. Confirmation: Validate the successful receipt of serialized weight arrays on the server.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 5: Master Aggregation and Web Telemetry

Focus: The server-side mathematical synthesis of the model and providing actionable business intelligence to the ML Engineer.

Story 5.1: As an ML Engineer, I want the central server to mathematically average the client weights, so that a new, smarter Master Model is created.

- **Acceptance Criteria:**
    - Given the aggregator receives weights from multiple participating clients, when the round ends, then it executes the Federated Averaging (FedAvg) algorithm.
    - Given the FedAvg completes, then the new global model is sent back to all clients for the next round.
- **Priority:** Must
- **Sizing:** L
- **3 C's:** Card: This story. Conversation: Verify the mathematical formulation of the FedAvg algorithm. Confirmation: Test the aggregation engine with simulated client arrays.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 5.2: As a Federation Owner, I want to view global versus client-specific metrics on my React frontend dashboard, so that I can monitor data drift, accuracy, and fairness.

- **Acceptance Criteria:**
  - Given the database is updated with round data, when the Web UI is loaded, then current global accuracy and loss are displayed.
  - Given anomalous client behavior occurs (e.g., outlier updates), then the dashboard triggers warnings or alerts.
- **Priority:** Must
- **Sizing:** M
- **3 C's:** Card: This story. Conversation: Design the visual layout for fairness indicators and accuracy charts. Confirmation: Validate UI charts reflect the correct database numbers.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.